先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 14 – Ensemble Learning II: Boosting, Gradient Boosting, and Stacking

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods: reweight, fit gradients, combine

Period	Main question	Worked example
1: 45 min	How does AdaBoost correct mistakes?	Five labels, three weighted rounds
2: 45 min	What should the next tree predict?	Residual regression and log-odds classification
3: 45 min	How do tree systems and stacking help?	Leaf gain and out-of-fold predictions

Link to Lecture 13. Forests mainly stabilize perturbed peers. Boosting changes the next learner's task using the current ensemble.

Learning target. Reconstruct the training signal, numerical update, and validation boundary for an ensemble, rather than select a method by its name.

The new learner adds a correction to the current score

Idea

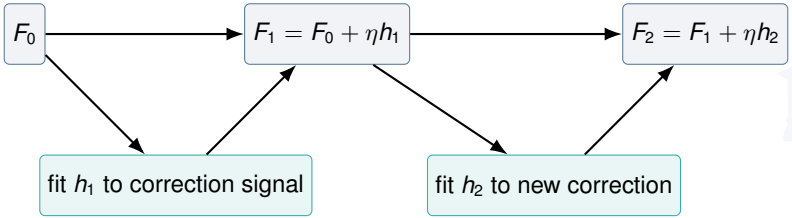
An additive ensemble starts from a simple score F_0 and adds one learner at a time. Earlier learners stay fixed while the next learner is fitted.

Work it out

$$F_m(x) = F_{m-1}(x) + \alpha_m h_m(x), \qquad F_M(x) = F_0(x) + \sum_{m=1}^{M} \alpha_m h_m(x).$$

What this tells us. If $F_0(x) = 2$, $h_1(x) = 3$, $\alpha_1 = 0.2$, $h_2(x) = -1$, and $\alpha_2 = 0.5$, the score becomes $2 + 0.6 - 0.5 = 2.1$. The sign and size of each correction matter. Classification may transform the score before making a decision.

The new learner adds a correction to the current score



A correction value is not an independent final prediction

Understand

Start a repair-cost model at 6. The first correction is -3 and the second is -1, both with coefficient 1. These are increments to the existing estimate.

Work through the example

$$F_2 = 6 - 3 - 1 = 2.$$

Interpretation. Averaging the three numbers would give $2/3$, but would combine a cost estimate with two residual corrections as if they had the same role. Additive models preserve the earlier score and add trained increments. The meaning of a learner's target must be clear before its output is combined.

A weak classifier must beat chance under current weights

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Idea

Use labels $y_i \in \{-1, +1\}$ and predictions $h_m(x_i) \in \{-1, +1\}$. Observation weights $w_{m,i}$ are nonnegative and sum to one.

Work it out

$$\epsilon_m = \sum_i w_{m,i} \mathbf{1}[y_i \neq h_m(x_i)].$$

What this tells us. Initially $w_{1,i} = 1/N$. In later rounds, raw mistake counts are insufficient: one high-weight mistake may matter more than several low-weight mistakes. Binary AdaBoost requires $0 < \epsilon_m < 0.5$ for a useful finite positive coefficient; a decision stump is a common learner.

Give the five cases features that generate the three stumps

Understand

The signed labels are $(+, +, -, -, +)$. Three illustrative measurements let the weak learners in our running example be implemented as real one-split classifiers.

Work through the example

Case	Temperature	Vibration	Load	y
A	20	1	8	+
B	22	8	1	+
C	5	2	2	-
D	25	3	3	-
E	30	9	9	+

Interpretation. Stumps. Predict negative below temperature 10, vibration 5, or load 5, and positive otherwise. The resulting mistake sets are D, A, and B respectively. Later reweighting decides which stump is useful at each round.

Five cases: start with equal observation weights

Idea

The five labels and the first weak classifier are fixed below. We use this same example for three rounds, so every later weight and score can be checked.

Work it out

Case	A	B	C	D	E
Label y	$+$	$+$	$-$	$-$	$+$
Classifier h_1	$+$	$+$	$-$	$+$	$+$
Initial weight	0.2	0.2	0.2	0.2	0.2

What this tells us. Only D is wrong, so $\epsilon_1 = 0.2$. The first classifier is 80% accurate under equal weights. The next question is how much trust to assign to h_1, and how much extra attention D should receive.

Before reweighting, different stumps can tie

Understand

Under initial weights 0.2 per case, the temperature stump misses D, the vibration stump misses A, and the load stump misses B. Each has the same weighted error.

Work through the example

Stump	Mistake	Error under initial weights
Temperature < 10	D	0.2
Vibration < 5	A	0.2
Load < 5	B	0.2

Interpretation. Selecting temperature first is a valid tie-breaking choice for this teaching example. AdaBoost does not require every round to have a unique best learner. Once weights change, the same candidate predictions can receive different weighted errors; that is how the training objective changes without changing the labels.

Practice: a new distribution changes the best candidate

Understand

After round 1, weights are $(0.125, 0.125, 0.125, 0.5, 0.125)$. Re-evaluate the same three one-split classifiers rather than recomputing raw accuracy.

Work through the example

Stump	Wrong case	Current weighted error
Temperature	D	0.500
Vibration	A	0.125
Load	B	0.125

Interpretation. Temperature now performs at weighted chance level and receives coefficient zero. Vibration and load remain useful and tied; choose vibration for round 2. All three still miss one of five raw cases. The algorithm's attention, rather than its observed labels, has changed.

Classifier confidence follows its weighted error

Idea

For the binary ± 1 convention, assign the weak classifier a coefficient based on its current weighted error.

Work it out

$$\alpha_m = \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}, \qquad \alpha_1 = \frac{1}{2} \log \frac{0.8}{0.2} = 0.6931.$$

What this tells us. A lower weighted error receives a larger positive coefficient. At error 0.5, the coefficient is zero. This is a score contribution, not a class probability. The factor $1/2$ is tied to $F = \sum_m \alpha_m h_m$ with binary signed labels; do not mix conventions from other variants.

The coefficient changes continuously with weighted error

Understand

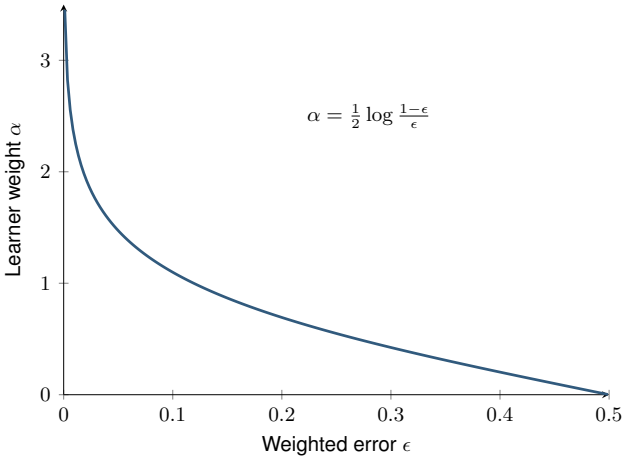
In our signed binary convention, $\alpha = \frac{1}{2}\log[(1-\epsilon)/\epsilon]$. Compare three useful weighted errors.

Work through the example

Weighted error	Coefficient	Meaning
0.49	0.0200	Only just beats chance
0.30	0.4236	Moderate score contribution
0.10	1.0986	Larger trusted contribution

Interpretation. These are weights in an additive decision score, not probabilities. For example, $\epsilon = 0.1$ gives $\alpha = \frac{1}{2}\log 9$. A very small error can yield a large coefficient, so finite-data noise and numerical handling near zero need attention rather than blind interpretation as certainty.

The coefficient changes continuously with weighted error



Increase mistake weights and explicitly normalize

Idea

Update each weight according to whether its signed prediction is correct, then divide by the normalization constant Z_m.

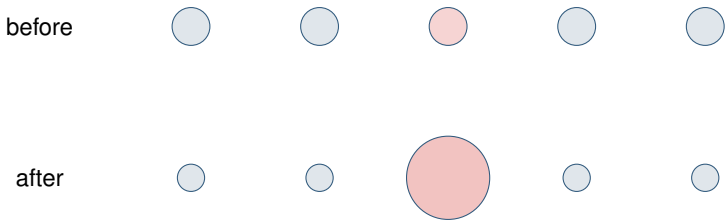
Work it out

$$w_{m+1,i} = \frac{w_{m,i} e^{-\alpha_m y_i h_m(x_i)}}{Z_m}, \qquad Z_m = \sum_i w_{m,i} e^{-\alpha_m y_i h_m(x_i)}.$$

Round 1 result	Correct A,B,C,E	Wrong D
Unnormalized weight	$0.2e^{-0.6931} = 0.1$	$0.2e^{0.6931} = 0.4$
Normalized: $Z_1 = 0.8$	0.125 each	0.500

What this tells us. The weights again sum to one. D now receives half of the total training attention; every correctly classified case receives one eighth.

Increase mistake weights and explicitly normalize



Mistakes gain relative weight; renormalize the complete distribution.

Why the previous mistakes receive half the new mass

Understand

With the exact optimal binary coefficient, $e^{\alpha} = \sqrt{(1-\epsilon)/\epsilon}$ and $Z = 2\sqrt{\epsilon(1-\epsilon)}$. Sum the updated weights over the cases the current learner missed.

Work through the example

$$\sum_{i:\, h_i \neq y_i} w_i^{new} = \frac{\epsilon e^{\alpha}}{Z} = \frac{1}{2}.$$

Interpretation. For $\epsilon = 0.2$, unnormalized mistake mass is 0.4 and total mass 0.8, giving 0.5. This property follows from the unshrunk update and coefficient convention. It does not say half of the observations are mistakes, or that every AdaBoost variant follows exactly the same identity.

Round 2 repairs D but makes a different mistake

Idea

Under weights $(0.125, 0.125, 0.125, 0.5, 0.125)$, the second classifier predicts $(-, +, -, -, +)$. It fixes D and misses A.

Work it out

$$\epsilon_2 = w_{2,A} = 0.125, \qquad \alpha_2 = \tfrac{1}{2}\log(0.875/0.125) = 0.9730.$$

What this tells us. Both weak classifiers miss only one of five cases, yet h_2 earns a larger coefficient. Its mistake is on a low-weight case, while it corrects the high-weight case D. AdaBoost judges the current weighted task, not the unweighted count of mistakes.

A one-mistake learner may be useful or useless in round 2

Understand

Compare two candidates after the first update: candidate U misses A and candidate V misses D. Each misses one of five observations.

Work through the example

$$\epsilon_U = 0.125 \Rightarrow \alpha_U = \tfrac{1}{2}\log 7 = 0.9730.$$
$$\epsilon_V = 0.5 \Rightarrow \alpha_V = 0.$$

Interpretation. U fixes the heavily weighted case D, while V repeats the current unresolved error. Raw 80% accuracy cannot distinguish them. Weighted errors encode the present ensemble's needs. This is why a sequence of weak learners is not the same as taking the globally strongest independent members and averaging them.

The new attention distribution changes again

Idea

Using $\alpha_2 = 0.9730$, a mistake is multiplied by $\sqrt{7}$ and a correct prediction by $1/\sqrt{7}$. Normalize the five weights.

Work it out

Case	A	B	C	D	E
After round 1	0.125	0.125	0.125	0.500	0.125
After round 2	0.500	0.0714	0.0714	0.2857	0.0714

What this tells us. A now holds half the mass; D still matters more than the other correct cases because it entered the round with greater weight. Weight updates depend on both the prior distribution and the current mistake, not only on a yes/no record of the latest prediction.

Compute the third-round update using exact fractions

Understand

Before round 3, weights are $(1/2, 1/14, 1/14, 2/7, 1/14)$ and the load stump misses only B. The new incorrect mass becomes 1/2.

Work through the example

Case	Before	After round 3
A	$1/2$	$7/26$
B	$1/14$	$1/2$
C	$1/14$	$1/26$
D	$2/7$	$2/13$
E	$1/14$	$1/26$

Interpretation. Correct-case weights are multiplied overall by $1/[2(1-1/14)] = 7/13$ after normalization. The new fractions sum to one. Attention shifts toward B, but D retains more mass than C or E because its prior weight was greater.

Round 3 completes a complementary set of learners

Idea

The third classifier predicts $(+, -, -, -, +)$, so only B is wrong under the current weights. Its weighted error is $1/14$.

Work it out

$$\alpha_3 = \frac{1}{2} \log \frac{13/14}{1/14} = \frac{1}{2} \log 13 = 1.2825.$$

Case	A	B	C	D	E
h_1	$+$	$+$	$-$	$+$	$+$
h_2	$-$	$+$	$-$	$-$	$+$
h_3	$+$	$-$	$-$	$-$	$+$

What this tells us. The learners make different mistakes. Their coefficients are determined in the distribution each encountered; the final decision uses the weighted sum, not the last learner alone.

Apply the trained ensemble to a genuinely new case

Understand

For a new device, suppose the three stumps predict $(+, -, -)$. Reuse the trained coefficients without computing new observation weights.

Work through the example

$$(\alpha_1, \alpha_2, \alpha_3) = (0.6931, 0.9730, 1.2825).$$

$$F(x) = 0.6931 - 0.9730 - 1.2825 = -1.5624 \Rightarrow \hat{y} = -1.$$

Interpretation. For another device with $(+, -, +)$, the score is 1.0026 and the decision is positive. Prediction does not need the new label, nor a new sample-weight distribution. The signed score orders evidence under this model but is not automatically a calibrated failure probability.

Combine the coefficients before taking the sign

Idea

Use $F = 0.6931h_1 + 0.9730h_2 + 1.2825h_3$. Predict positive when $F > 0$ and negative when $F < 0$.

Work it out

Case	Weighted score F	True label	Decision
A	$0.6931 - 0.9730 + 1.2825 = 1.0026$	$+$	Correct
B	$0.6931 + 0.9730 - 1.2825 = 0.3836$	$+$	Correct
D	$0.6931 - 0.9730 - 1.2825 = -1.5624$	$-$	Correct

What this tells us. C has score -2.9486 and E has $+2.9486$, also correct. Every member made one mistake, but this ensemble makes none on these five training cases. That is a training result, not a guaranteed test result.

Weighted voting can differ from counting classifiers

Understand

Consider a separate illustrative additive classifier with positive coefficients $(0.2, 0.3, 1.0)$. Its three weak predictions on one case are $(+, +, -)$.

Work through the example

$$F = 0.2 + 0.3 - 1.0 = -0.5.$$

Interpretation. Unweighted majority predicts positive, but the trained weighted score predicts negative. This is not the same coefficient set as the five-case running example. Coefficients retain relative confidence learned under each round's distribution. Before interpreting any vote, specify whether it counts labels, averages probabilities, or adds signed score contributions.

Exponential loss pays attention to signed margins

Idea

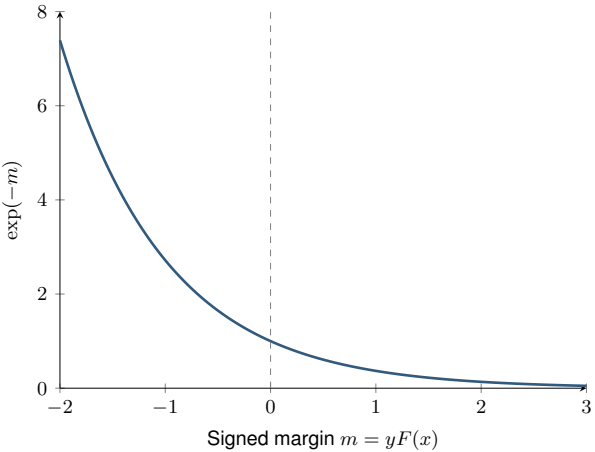
The signed margin $m_i = y_i F(x_i)$ is positive for a correct decision and negative for a wrong decision. Exponential loss is $L_i = e^{-m_i}$.

Work it out

Margin	Decision	Exponential loss
-1	Wrong	$e^{1} = 2.718$
0	On the boundary	1.000
$+1$	Correct	$e^{-1} = 0.368$
$+3$	Correct, larger margin	$e^{-3} = 0.050$

What this tells us. A confidently wrong case has a large loss, while an already secure correct case has little influence. This explains attention redistribution and also why mislabeled or irreducibly difficult cases can dominate later rounds.

Exponential loss pays attention to signed margins



A correct low-margin case still has much larger loss

Understand

In the running ensemble, positive case B has margin 0.3836, while positive case E has margin 2.9486. Both decisions are correct.

Work through the example

$$L_B = e^{-0.3836} \approx 0.6814, \qquad L_E = e^{-2.9486} \approx 0.0524.$$

Interpretation. B carries about 13 times E's exponential loss at this score. A zero training classification error does not make every sample equally secure or make exponential loss zero. Continued rounds can change these margins, but validation evidence must decide whether doing so improves unseen predictions.

A contradictory label produces a large exponential penalty

Understand

Hold the current score fixed at $F = 2.9486$. A positive recorded label gives margin $+2.9486$; flipping that recorded label to negative gives margin -2.9486.

Work through the example

$$L_+ = e^{-2.9486} \approx 0.0524, \qquad L_- = e^{2.9486} \approx 19.08.$$

Interpretation. The same score receives a dramatically different learning signal when the recorded label changes. This illustrates sensitivity to mislabeled observations; it does not identify which labels are actually wrong. Inspect persistently difficult cases using domain evidence rather than deleting them merely because the model disagrees.

The coefficient minimizes the current exponential loss

Idea

With the learner fixed and normalized current weights, correct cases have total mass $1 - \epsilon$, and mistakes have mass ϵ. The round objective becomes

Work it out

$$Z(\alpha) = (1-\epsilon)e^{-\alpha} + \epsilon e^{\alpha}.$$

$$Z'(\alpha) = -(1-\epsilon)e^{-\alpha} + \epsilon e^{\alpha} = 0 \Rightarrow \alpha = \tfrac{1}{2}\log\frac{1-\epsilon}{\epsilon}.$$

What this tells us. Substitution gives $Z^* = 2\sqrt{\epsilon(1-\epsilon)}$. At $\epsilon = 0.2$, $Z^* = 0.8$, exactly the normalization value in our first round. The learner minimizes weighted errors; the coefficient balances correct and incorrect exponential contributions.

Check the loss minimum, not only the derivative equation

Understand

For weighted error 0.2, the one-round objective is $Z(\alpha) = 0.8e^{-\alpha} + 0.2e^{\alpha}$. Its second derivative is positive everywhere.

Work through the example

$$Z''(\alpha) = 0.8e^{-\alpha} + 0.2e^{\alpha} > 0.$$

Coefficient	Objective Z
0	1.000
0.6931	0.800
1.4	approximately 1.008

Interpretation. The stationary point is the unique minimum. A much larger coefficient can worsen this objective even for a useful weak classifier. Confidence weighting has a specific optimum; it is not a rule to give every accurate learner an arbitrarily large score.

Lower training loss does not certify lower test error

Idea

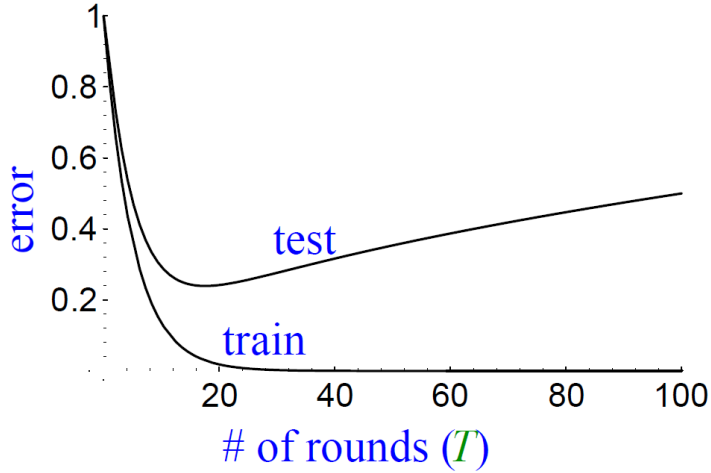
With the exact binary AdaBoost updates and initially uniform weights, the average training exponential loss equals $\prod_m Z_m$. Training classification error is bounded above by this loss.

Work it out

$$\frac{1}{N}\sum_i \mathbf{1}[y_i F(x_i) \le 0] \;\le\; \frac{1}{N}\sum_i e^{-y_i F(x_i)} = \prod_m 2\sqrt{\epsilon_m(1-\epsilon_m)}.$$

What this tells us. When each useful round has $\epsilon_m < 0.5$, the bound decreases. It concerns the training sample. Noise, finite data, and excessive capacity can still harm validation performance; choose the number of rounds using held-out evidence.

Lower training loss does not certify lower test error



Calculate the three-round exponential-loss bound

Understand

The running example has weighted errors $1/5$, $1/8$, and $1/14$. Multiply the corresponding normalization factors.

Work through the example

$$Z_1 = 0.8, \quad Z_2 = 2\sqrt{(1/8)(7/8)} = 0.6614, \quad Z_3 = \sqrt{13}/7 = 0.5151.$$

$$\prod_{m=1}^{3} Z_m \approx 0.2725.$$

Interpretation. All five training cases are classified correctly, so training error 0 is below the exponential-loss bound 0.2725. A nonzero upper bound is compatible with zero observed errors. The product describes this exact training update; it is not an upper bound on a new-device test error without additional assumptions.

Weakness, noisy labels, and stopping belong together

Idea

A decision stump limits each correction to one split. Deeper weak trees can model interactions but may also chase difficult or mislabeled training cases more aggressively.

Work it out

Round count	Training error	Validation error
10	0.12	0.16
50	0.02	0.11
200	0.00	0.14

What this tells us. Choose 50 rounds in this illustration, rather than maximizing training fit. Shrinkage may temper each correction, but its precise update depends on the AdaBoost variant. Our hand calculation used unshrunk coefficients; gradient boosting next provides a general loss-based correction framework.

Shrinkage changes the exact reweighting identity

Understand

As an illustrative one-step modification, use half the classic coefficient for error 0.2: $\alpha = 0.3466$. Apply the same signed exponential weight update and normalize.

Work through the example

$$\text{Wrong mass} = 0.2e^{0.3466} = 0.2828, \quad \text{Correct mass} = 0.8e^{-0.3466} = 0.5657.$$

$$\text{New wrong fraction} = 0.2828/(0.2828 + 0.5657) = 1/3.$$

Interpretation. The wrong fraction is not 1/2 because the exact optimal coefficient was reduced. This defined example explains why identities must be tied to an update convention. Software variants need their own definitions; do not mix a shrinkage setting with an unshrunk proof.

A regression correction predicts what is still missing

Idea

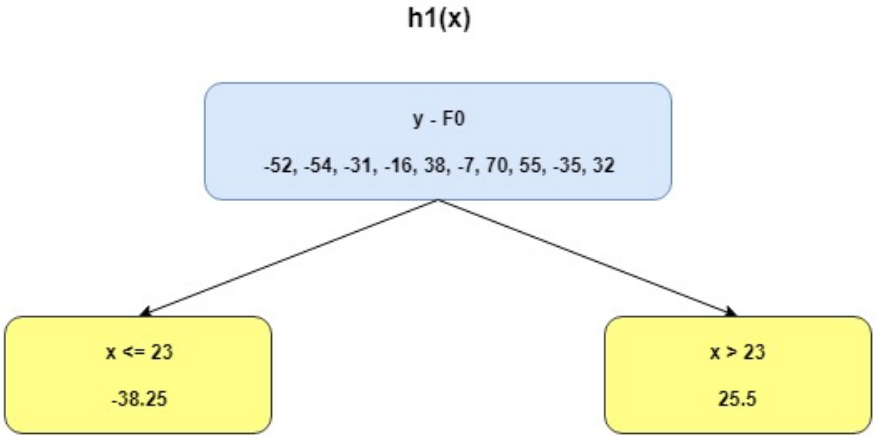
For squared-error regression, $L(y, F) = \frac{1}{2}(y - F)^2$. Its negative derivative with respect to the current prediction is the residual.

Work it out

$$-\frac{\partial L}{\partial F} = y - F = r.$$

What this tells us. If the true repair costs are $(2, 4, 8, 10)$ and current predictions are $(6, 6, 6, 6)$, residuals are $(-4, -2, 2, 4)$. The next regression tree fits these residuals, not the original costs. A negative tree output decreases an overestimate; a positive output increases an underestimate.

Reference example: a regression tree fits the current residuals



Loss scaling changes gradient magnitude, not its direction

Understand

At true cost 10 and current prediction 6, compare $L_1 = \frac{1}{2}(y - F)^2$ with $L_2 = (y - F)^2$. They have the same minimizers but different derivatives.

Work through the example

$$-\partial L_1/\partial F = 4, \qquad -\partial L_2/\partial F = 8.$$

Interpretation. If learners reproduce these directions exactly, using learning rate 0.1 for the first or 0.05 for the second gives the same increment 0.4. State the loss convention before copying gradients. Tree fitting, line searches, and regularization can affect the full procedure, so scaling is not always a harmless software substitution.

Start regression from the best constant prediction

Idea

For squared loss, the constant that minimizes training loss is the target mean. Let inputs $x = (1, 2, 3, 4)$ and targets $y = (2, 4, 8, 10)$.

Work it out

$$F_0(x) = \bar{y} = \frac{2 + 4 + 8 + 10}{4} = 6.$$

x	1	2	3	4
y	2	4	8	10
F_0	6	6	6	6
Residual r_1	-4	-2	2	4

$$\mathrm{MSE}(F_0) = (16 + 4 + 4 + 16)/4 = 10.$$

What this tells us. The initial model has no feature dependence. The first tree asks whether the residuals have a predictable pattern in x.

A different loss changes the best constant initialization

Understand

Consider targets $(2, 4, 8, 30)$, including one large cost. Squared loss prefers the mean 11; absolute loss prefers any median between 4 and 8, such as 6.

Work through the example

$$\sum_i |y_i - 11| = 9 + 7 + 3 + 19 = 38, \quad \sum_i |y_i - 6| = 4 + 2 + 2 + 24 = 32.$$

Interpretation. The starting model depends on the chosen loss, not a universal rule to use the mean. Absolute loss is nondifferentiable at zero, requiring an appropriate subgradient or specialized treatment. This separate example introduces robustness; it does not alter the earlier squared-loss targets $(2, 4, 8, 10)$.

The first tree fits two residual means

Idea

Split at $x = 2.5$. A least-squares regression leaf predicts the mean residual among its training cases.

Work it out

$$h_1(x) = \begin{cases} (-4-2)/2 = -3, & x \le 2.5, \\ (2+4)/2 = 3, & x > 2.5. \end{cases}$$

x	1	2	3	4
h_1	-3	-3	3	3
$F_1 = F_0 + h_1$	3	3	9	9
New residual	-1	1	-1	1

What this tells us. With learning rate 1, MSE falls from 10 to 1. The tree learned a useful broad correction but cannot yet distinguish cases within either leaf.

Why the first residual tree chooses the middle split

Understand

The first squared-loss residuals are $(-4, -2, 2, 4)$ at inputs $(1, 2, 3, 4)$. Compare candidate thresholds by within-leaf residual SSE.

Work through the example

Threshold	Leaf residual means	Residual SSE
1.5	-4 and $4/3$	$56/3 \approx 18.67$
2.5	-3 and +3	4
3.5	$-4/3$ and +4	$56/3 \approx 18.67$

Interpretation. The threshold 2.5 best captures the broad residual pattern. Its leaf values become corrections to the current constant prediction. This criterion fits the current residual targets; it is not a new tree fit to the original repair costs without reference to the ensemble.

Shrinkage changes how much of the correction is used

Idea

Use the same fitted tree, but apply learning rate $\eta = 0.5$. The update is $F_1 = F_0 + \eta h_1$, so only half the proposed correction is added.

Work it out

x	1	2	3	4
F_0	6	6	6	6
$0.5h_1$	-1.5	-1.5	1.5	1.5
F_1	4.5	4.5	7.5	7.5
Residual	-2.5	-0.5	0.5	2.5

$$\text{MSE} = \frac{6.25 + 0.25 + 0.25 + 6.25}{4} = 3.25.$$

What this tells us. One smaller step improves less than a full step on training data, but can provide a more conservative learning path. Smaller learning rates usually require more rounds; validate the tradeoff.

Compare three learning rates on exactly the same fitted tree

Understand

Keep $F_0 = 6$ and $h_1 = (-3, -3, +3, +3)$ fixed. Apply different fractions of this same correction, then compute target MSE.

Work through the example

Learning rate	Updated predictions	MSE
1.00	3, 3, 9, 9	1.0000
0.50	4.5, 4.5, 7.5, 7.5	3.2500
0.25	5.25, 5.25, 6.75, 6.75	6.0625

Interpretation. A smaller step improves less on this first training round and leaves more work for later trees. The comparison isolates step size; it does not prove the largest learning rate is best for validation. Each path must recompute its own next residuals.

The next tree must use the updated residuals

Idea

Continue the learning-rate-1 path, whose residuals are $(-1, 1, -1, 1)$. An illustrative second stump splits at $x = 1.5$.

Work it out

$$h_2(x) = \begin{cases} -1, & x \le 1.5, \\ (1 - 1 + 1)/3 = 1/3, & x > 1.5. \end{cases}$$

x	1	2	3	4
F_1	3	3	9	9
F_2	2	3.333	9.333	9.333

$$\mathrm{MSE}(F_2) = \frac{0 + (2/3)^2 + (-4/3)^2 + (2/3)^2}{4} = 2/3.$$

What this tells us. The second tree fits what remains after the first tree. Reusing the original residuals would add the same broad correction again and overshoot.

Predict a new input by walking through every correction tree

Understand

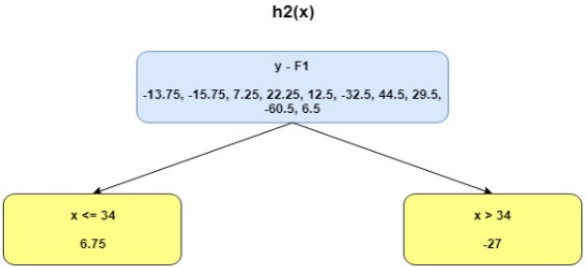
Use the two-tree squared-loss model: $F_0 = 6$, $h_1 = -3$ below 2.5 and $+3$ above it; $h_2 = -1$ below 1.5 and $+1/3$ above it.

Work through the example

$$x = 2.2: \; F_2 = 6 - 3 + 1/3 = 3.333.$$
$$x = 2.8: \; F_2 = 6 + 3 + 1/3 = 9.333.$$

Interpretation. New-case prediction does not require knowing its residual. Route the original input through the stored trees, then add their outputs. The resulting function is piecewise constant and can jump at a threshold. A fitted tree sum need not express a smooth trend or reliable extrapolation.

Read a residual tree from the Chinese reference



Read the figure. The blue box contains the current salary residuals $y - F_1$. The split uses years of experience $x \leq 34$. The two yellow leaves predict mean residuals: $+6.75$ and -27.

Takeaway. These leaf values are corrections to the existing salary prediction, not salary predictions by themselves. With $\eta = 0.1$, the added amounts would be $+0.675$ and -2.7. Recompute residuals after applying the update.

A globally useful correction can worsen an individual case

Understand

The salary reference tree adds $+6.75$ for experience at most 34 years and -27 above 34. Apply it to two rows from that teaching example.

Work through the example

Years	Salary y	Current F_1	After F_2
5	82	95.75	102.50
40	166	159.50	132.50

Interpretation. For the five-year case, absolute residual increases from 13.75 to 20.50; for the forty-year case it increases from 6.50 to 33.50. Leaf means optimize aggregate squared loss over their groups, not every observation individually. Judge the overall objective while inspecting which patterns remain.

Other losses supply other pseudo-residuals

Idea

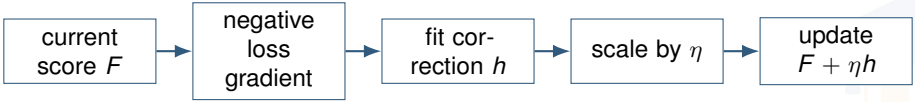
For a differentiable loss, compute one negative derivative per observation at the current ensemble prediction. Fit a regression learner to these pseudo-residuals.

Work it out

$$r_{m,i} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)}, \qquad F_m = F_{m-1} + \eta h_m.$$

What this tells us. Squared loss gives $y - F$. Other losses give different targets, and leaf values may be refined by minimizing the original loss within each leaf. Gradient boosting approximates a useful function-space descent direction using a restricted learner, rather than directly changing one global parameter vector.

Other losses supply other pseudo-residuals



Huber loss caps the gradient influence of a large residual

Understand

For Huber loss with transition $\delta = 1$, the negative derivative with respect to the prediction is the clipped residual.

Work through the example

$$r_{\text{pseudo}} = \text{clip}(y - F, -1, 1).$$

Interpretation. A residual 0.4 gives pseudo-residual 0.4; a residual 10 gives 1 rather than 10; a residual -10 gives -1. Small errors use a squared-loss direction, while large errors have bounded first-order influence. This changes the learner's training targets; final leaf optimization and step size still follow the chosen algorithm.

Binary boosting updates log-odds, not a probability directly

Idea

For labels $y \in \{0, 1\}$, let $F(x)$ be a log-odds score and transform it to probability by $p = \sigma(F)$. Use binary cross-entropy.

Work it out

$$p = \frac{1}{1 + e^{-F}}, \qquad L(y, F) = -y \log p - (1 - y) \log(1 - p).$$

$$\frac{\partial L}{\partial F} = p - y, \qquad r = y - p.$$

What this tells us. A tree correction is added to F, after which probability is recomputed. Adding a tree directly to p can leave the interval $[0, 1]$ and does not implement this loss-based update.

Verify that a score update actually lowers classification loss

Understand

For a positive case initially predicted with $p = 0.2$, the log-odds score is $F = -1.3863$. Add a fitted correction of 0.8 with learning rate 0.1.

Work through the example

$$F_{new} = -1.3063, \qquad p_{new} = \sigma(F_{new}) \approx 0.2131.$$
$$L_{old} = -\log 0.2 = 1.6094, \qquad L_{new} \approx 1.5457.$$

Interpretation. The positive correction moves the score and probability upward and reduces the case's cross-entropy. It does not add 0.08 directly to the probability. This local numerical check connects the gradient sign to the stated loss objective.

Calculate a classification correction direction

Idea

Case A is positive with predicted probability 0.2; case B is negative with probability 0.8. Their pseudo-residuals push in opposite directions.

Work it out

Case	y	p	Negative gradient $y - p$
A	1	0.2	+0.8
B	0	0.8	-0.8

$$F_A = \log(0.2/0.8) = -1.386.$$

$$F_A^{new} = F_A + 0.1(0.8) = -1.306, \qquad p_A^{new} \approx 0.213.$$

What this tells us. This illustrative fitted correction moves the positive case toward higher probability. The actual tree shares leaf values across cases, so it cannot necessarily reproduce every pseudo-residual exactly.

A leaf shares the mean classification-gradient target

Understand

A regression-tree leaf contains labels $(1, 1, 0)$ and current probabilities $(0.2, 0.4, 0.6)$. Their first-order pseudo-residuals are $(0.8, 0.6, -0.6)$.

Work through the example

$$h_{leaf} = (0.8 + 0.6 - 0.6)/3 = 0.2667.$$

Interpretation. With learning rate 0.1, every case in the leaf receives a log-odds increment 0.02667. Positive cases move in a helpful direction; the negative case moves slightly upward too. The tree approximates the aggregate gradient using shared regions. Some algorithms subsequently refine leaf values under the original loss.

Multiclass gradients use one score for each class

Idea

For K classes, encode the label as a one-hot vector and maintain scores $F_1, \ldots, F_K$. Softmax gives probabilities.

Work it out

$$p_k = \frac{e^{F_k}}{\sum_{j=1}^{K} e^{F_j}}, \qquad L = -\sum_k y_k \log p_k, \qquad r_k = y_k - p_k.$$

What this tells us. For true class 2, calculate

$$y = (0, 1, 0), \quad p = (0.2, 0.5, 0.3), \quad r = (-0.2, +0.5, -0.3).$$

The true class score rises; competing scores fall. Tree systems typically fit class-specific corrections, then recompute all probabilities together.

Take one multiclass score step and recompute softmax

Understand

For true class 2, start with $p = (0.2, 0.5, 0.3)$ and choose scores $F_k = \log p_k$. The pseudo-residual vector is $(-0.2, 0.5, -0.3)$.

Work through the example

$$F_{new} = F + 0.2r, \qquad e^{F_{new}} = (0.2e^{-0.04}, 0.5e^{0.10}, 0.3e^{-0.06}).$$
$$p_{new} \approx (0.1871, 0.5379, 0.2750).$$

Interpretation. The true-class probability increases from 0.5 to 0.538, and its loss decreases from 0.693 to about 0.620. Recompute all classes using a shared denominator. The example assumes the correction learner reproduces this vector at the case.

Initialize classification from the training class rate

Idea

A constant binary model that minimizes cross-entropy predicts the training positive fraction π. In the log-odds convention, its starting score is $F_0 = \log[\pi/(1-\pi)]$.

Work it out

$$\pi = 0.10 \Rightarrow F_0 = \log(0.1/0.9) = -2.197, \qquad \sigma(F_0) = 0.10.$$

What this tells us. Starting every score at zero implies a 0.5 probability in binary classification. That can be a valid convention in some formulations, but it does not express an imbalanced class prior. Use training data only to estimate the initial rate; later trees learn feature-dependent departures from it.

A prior-optimal constant can have zero total gradient and useful splits

Understand

Among 50 training cases, five are positive, so initialize $p = 0.1$. Positive pseudo-residuals are 0.9 and negative ones are -0.1.

Work through the example

$$\sum r = 5(0.9) + 45(-0.1) = 0.$$

Interpretation. A ten-case region with four positives has residual sum $4(0.9) + 6(-0.1) = 3$ and mean 0.3. The other 40 cases sum to -3 and mean -0.075. A constant correction would do nothing, but a feature-dependent tree can raise one region and lower another. Global balance does not imply no learnable structure.

Different controls restrain different parts of boosting

Control	What changes	Practical implication
Learning rate	Contribution per tree	Smaller steps usually need more rounds
Tree depth / leaves	Complexity of each correction	Limits interactions and local detail
Row subsampling	Cases fitted in a round	Adds randomness to the learning path
Feature subsampling	Candidates considered	Limits repeated feature use
Early stopping	Total number of corrections	Select using validation loss

Connect the controls. A small learning rate with very few trees can underfit. Very deep trees with many rounds can chase noise. Tune the joint learning path on a task-appropriate validation split.

Row subsampling perturbs a sequential correction, not the target story

Understand

The residuals are $(-4, -2, 2, 4)$ at inputs 1 through 4. In one illustrative round, fit only cases 1 and 4, using a split between them.

Work through the example

$$h = (-4, -4, +4, +4), \qquad F_0 + 0.5h = (4, 4, 8, 8).$$

Interpretation. The MSE on all four original targets is $(4 + 0 + 0 + 4)/4 = 2$. Full-data leaf means with the same step gave MSE 3.25. This particular subsample happens to help; it is not a universal advantage. Subsampling adds path randomness while every new tree still learns the current ensemble's correction target.

Tree depth controls which interactions can be corrected

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Idea

A decision stump changes its output according to one feature threshold. Adding stumps can build additive effects, but a single stump cannot condition a vibration correction on temperature.

Work it out

- Additive pattern: risk rises with temperature and separately with age.
- Interaction pattern: high vibration is dangerous only when temperature is also high.
- A two-level tree can first test temperature and then vibration in one branch.

What this tells us. Use deeper corrections when the task needs interactions, but validate their benefit. Depth is a capacity control, whereas learning rate scales corrections after they have been fitted.

An additive collection of stumps cannot express every interaction

Understand

For binary inputs, consider targets 0, 1, 1, 0 on $(0,0), (1,0), (0,1), (1,1)$. A sum of single-feature stumps is an additive function $a + bx_1 + cx_2$ on these inputs.

Work through the example

$$a = 0, \quad a + b = 1, \quad a + c = 1 \Rightarrow a + b + c = 2 \neq 0.$$

Interpretation. Matching the first three points prevents matching the fourth. A two-level tree can condition its second split on the first feature and represent this interaction. The exercise concerns correction capacity: reducing learning rate alone does not give additive stumps a missing interaction structure.

Early stopping selects the learning path using validation

Idea

Inspect validation loss after successive boosting rounds. The training loss may continue falling after unseen-data performance stops improving.

Work it out

Round	Training loss	Validation loss
20	0.42	0.46
60	0.27	0.34
100	0.18	0.32
160	0.10	0.36

What this tells us. Choose the best supported round, about 100 in this illustration. A patience rule tolerates a limited number of rounds without improvement. Preserve the best model, rather than automatically taking the last round. The final test set evaluates the selected procedure; it must not drive repeated stopping decisions.

Practice: stop after patience expires and restore the best round

Understand

Use patience two, with lower validation loss considered better. Inspect five consecutive round losses; no improvement occurs after round 3.

Work through the example

Round	Validation loss	Best round so far
1	0.340	1
2	0.330	2
3	0.320	3
4	0.325	3
5	0.326	3

Interpretation. At round 5, two rounds without improvement have occurred. Stop and retain round 3 rather than round 5. The patience rule manages fluctuations but cannot predict every future improvement. Its validation data is part of model selection, so the final test set remains outside these stopping decisions.

A compact implementation follows the squared-loss story

```
from sklearn.ensemble import GradientBoostingRegressor
gb = GradientBoostingRegressor(
    loss="squared_error", n_estimators=300,
    learning_rate=0.05, max_depth=2,
    subsample=0.8, validation_fraction=0.2,
    n_iter_no_change=20, random_state=14)
gb.fit(X_train, y_train)
y_pred = gb.predict(X_test)
```

Read it as a process. The model starts from a constant, fits successive correction trees, uses small updates, and stops according to an internal validation holdout.

Boundary. This automatic random holdout suits independent rows; grouped or temporal tasks need an explicitly designed validation procedure.

Use a classifier interface for the log-loss model

Connect code to the classification story

Fit a binary log-loss boosted classifier, then identify which probability column represents label 1.

```
from sklearn.ensemble import GradientBoostingClassifier
clf = GradientBoostingClassifier(
    loss="log_loss", learning_rate=0.05,
    n_estimators=100, max_depth=2, random_state=14)
clf.fit(X_train, y_train)
col = list(clf.classes_).index(1)
p = clf.predict_proba(X_valid)[:, col]
```

Interpretation. The model accumulates score corrections and returns class probabilities. Validate log loss or the action metric on the held-out data. This is a different objective and interface from the squared-loss regressor, despite similar tree and learning-rate arguments.

XGBoost adds curvature and explicit tree regularization

Idea

Gradient boosting chooses corrections from a loss gradient. XGBoost uses a local second-order loss approximation and adds a penalty on the new tree's leaves and scores.

Work it out

$$\widetilde{J} = \sum_i \left[g_i h(x_i) + \tfrac{1}{2} H_i h(x_i)^2 \right] + \gamma T + \tfrac{\lambda}{2} \sum_{j=1}^{T} w_j^2 .$$

What this tells us. Here g_i is the first derivative, H_i the second derivative, T the number of leaves, and w_j a leaf score. Curvature influences how far to move; penalties discourage unnecessary leaves and extreme scores. Efficient search and missing-value handling are additional system features.

For squared loss, the second-order expansion is exact

Understand

At current prediction $F = 6$ and target $y = 2$, squared loss is $L = \frac{1}{2}(6-2)^2 = 8$, with gradient 4 and curvature 1. Propose correction $h = -1$.

Work through the example

$$L(F+h) = 8 + 4(-1) + \tfrac{1}{2}(1)(-1)^2 = 4.5.$$

Interpretation. Direct evaluation gives $\frac{1}{2}(5-2)^2 = 4.5$, matching the expansion. A correction $+1$ instead gives 12.5. The quadratic loss makes the Taylor expression exact; for general losses it is local approximation. This explains both the information in slope and curvature and the need to control large moves.

Compute both slope and curvature for a case

Idea

A local Taylor expansion uses $g = \partial L/\partial F$ and $H = \partial^2 L/\partial F^2$ at the current score. Terms independent of the new correction do not affect its optimization.

Work it out

Loss	First derivative g	Second derivative H
$\frac{1}{2}(F-y)^2$	$F-y$	1
Binary cross-entropy	$p-y$	$p(1-p)$

What this tells us. For a positive case with $p = 0.2$, $g = -0.8$ and $H = 0.16$. For squared loss with $F = 6$, $y = 2$, $g = 4$ and $H = 1$. The one-step direction still corrects error; curvature and regularization scale its size.

Equal curvature can accompany very different correction demands

Understand

For binary cross-entropy, gradient is $g = p - y$ and curvature is $H = p(1 - p)$. Compare confident right and confident wrong predictions.

Work through the example

y	p	g	H
1	0.1	-0.9	0.09
1	0.9	-0.1	0.09
0	0.1	+0.1	0.09
0	0.9	+0.9	0.09

Interpretation. The probabilities 0.1 and 0.9 have equal curvature but different signed errors. A positive label at probability 0.1 needs a strong upward direction; a negative label at 0.9 needs a strong downward direction. Curvature scales a proposed move, while gradient supplies its local direction and demand.

A fixed leaf combines its gradient statistics

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Idea

All observations in leaf j receive the same score w_j. Define $G_j = \sum_{i \in I_j} g_i$ and $S_j = \sum_{i \in I_j} H_i$. The leaf-dependent objective is quadratic.

Work it out

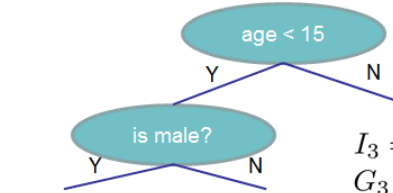
$$J_j(w_j) = G_j w_j + \tfrac{1}{2}(S_j + \lambda) w_j^2 + \gamma.$$

$$\frac{dJ_j}{dw_j} = G_j + (S_j + \lambda) w_j = 0 \Rightarrow w_j^* = -\frac{G_j}{S_j + \lambda}.$$

What this tells us. The score balances the summed slope against curvature plus regularization. We use S_j for the curvature sum to keep it distinct from the per-observation value H_i. Tree structure determines which observations share these statistics.

A fixed leaf combines its gradient statistics

Instance index		gradient statistics
1		g1, h1
2		g2, h2
3		g3, h3
4		g4, h4
5		g5, h5



$$I_1 = \{1\} \quad G_1 = g_1 \quad H_1 = h_1$$

$$I_2 = \{4\} \quad G_2 = g_4 \quad H_4 = h_4$$

$$I_3 = \{2, 3, 5\} \quad G_3 = g_2 + g_3 + g_5 \quad H_3 = h_2 + h_3 + h_5$$

$$Obj = -\sum_j \frac{G_j^2}{H_j + \lambda} + 3\gamma$$

The smaller the score is, the better the structure is

The regularized leaf formula contains the residual mean as a special case

Understand

For squared loss, $g_i = F_i - y_i = -r_i$ and $H_i = 1$. A leaf with n observations therefore has $G = -\sum r_i$ and curvature sum $S = n$.

Work through the example

$$w^* = \frac{\sum r_i}{n + \lambda}.$$

Interpretation. For residuals $(-4, -2)$, $n = 2$: with $\lambda = 0$, $w = -6/2 = -3$, the same mean used by the first gradient-boosting tree. With $\lambda = 2$, $w = -6/4 = -1.5$. The XGBoost expression extends rather than contradicts ordinary residual fitting.

Regularization shrinks a leaf correction: use numbers

Idea

Two positive cases share a leaf, each with current probability 0.2. Thus $G = -0.8 - 0.8 = -1.6$ and $S = 0.16 + 0.16 = 0.32$.

Work it out

$$\lambda = 0: \quad w^* = 1.6/0.32 = 5.$$
$$\lambda = 1: \quad w^* = 1.6/1.32 \approx 1.212.$$

What this tells us. The unregularized local approximation proposes a very large score shift. Adding curvature regularization reduces it. With learning rate 0.1, the applied update is about 0.121 rather than 1.212. Regularization and learning rate are distinct: one changes the fitted leaf optimum, the other scales the chosen correction.

Learning rate and leaf regularization are not globally interchangeable

Understand

Two settings can give the same update for one leaf but differ for another. Use $G = -6$ and compare curvature sums $S = 2$ and $S = 10$.

Work through the example

Setting	Update at $S = 2$	Update at $S = 10$
$\lambda = 0,\ \eta = 0.5$	$0.5(6/2) = 1.5$	$0.5(6/10) = 0.3$
$\lambda = 2,\ \eta = 1$	$6/4 = 1.5$	$6/12 = 0.5$

Interpretation. Learning rate multiplies every chosen leaf score; regularization enters each leaf's denominator and affects split scores too. Coincidence on one leaf does not make the two controls equivalent over a whole tree or boosting path.

A split must earn its extra leaf

Idea

Compare splitting one leaf into left and right leaves. The improvement in the optimal second-order objective is

Work it out

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{S_L+\lambda} + \frac{G_R^2}{S_R+\lambda} - \frac{(G_L+G_R)^2}{S_L+S_R+\lambda}\right] - \gamma.$$

What this tells us. Let $G_L = -4$, $S_L = 2$, $G_R = +4$, $S_R = 2$, $\lambda = 1$, and $\gamma = 1$. The unsplit gradient sum is zero; the gain is $\frac{1}{2}(16/3 + 16/3) - 1 = 4.333$. The split is beneficial under this approximation. If the gain is nonpositive, its fit improvement does not cover the added complexity cost.

Practice: score three candidate XGBoost splits

Understand

The parent has $G = 0$ and curvature sum 4. Set $\lambda = 1$, $\gamma = 1$. Each candidate has left and right curvature sums 2, but different gradient separation.

Work through the example

Candidate	Left / right G	Gain
A	-4 / +4	$\frac{1}{2}(16/3 + 16/3) - 1 = 4.333$
B	-2 / +2	$\frac{1}{2}(4/3 + 4/3) - 1 = 0.333$
C	-1 / +1	$\frac{1}{2}(1/3 + 1/3) - 1 = -0.667$

Interpretation. Choose A among these candidates. C does not cover the added leaf cost. Opposite gradients make separate corrections useful even though the parent sum is zero. The half factor and complexity cost belong to the declared objective.

Histogram bins reduce the split-search workload

Idea

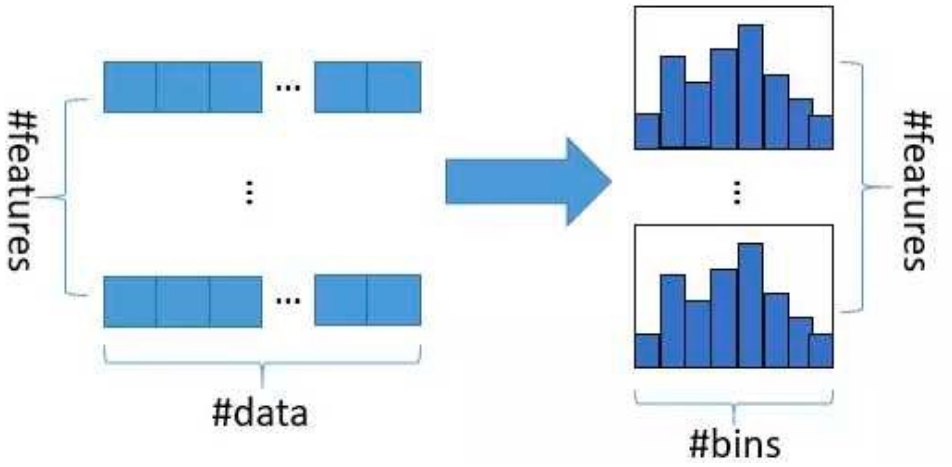
Exact split search sorts distinct feature values and checks many thresholds. Histogram methods map values into a limited number of bins and accumulate gradient and curvature sums per bin.

Work it out

Feature values	1.0, 1.1, 1.2	2.8, 2.9	5.0, 5.1
Bin	1	2	3
Stored statistics	$\sum g, \sum H$	$\sum g, \sum H$	$\sum g, \sum H$

What this tells us. Scan cumulative statistics across bin boundaries to calculate candidate gains. Fewer boundaries reduce memory and search work, while coarse bins may miss a useful threshold. Histogram search is used by modern tree systems, including LightGBM and histogram-based XGBoost variants.

Histogram bins reduce the split-search workload



Scan histogram prefix sums to choose a boundary

Understand

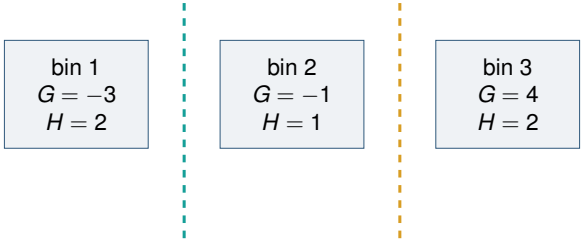
Three feature bins contain gradient sums $(-3, -1, 4)$ and curvature sums $(2, 1, 2)$. The parent gradient is zero. Use $\lambda = 1$ and $\gamma = 1$.

Work through the example

Boundary	Left (G, S)	Right (G, S)	Gain
After bin 1	(-3,2)	(3,3)	$\frac{1}{2}(9/3 + 9/4) - 1 = 1.625$
After bin 2	(-4,3)	(4,2)	$\frac{1}{2}(16/4 + 16/3) - 1 = 3.667$

Interpretation. Cumulative bin statistics produce both candidates without rescanning every original value. The second boundary wins in this example. Histogram approximation changes candidate locations and computation, while the loss-based gain formula remains the same.

Scan histogram prefix sums to choose a boundary



$\lambda = 1,\ \gamma = 1$: choose the second boundary.

Leaf-wise growth spends capacity where gain is largest

Idea

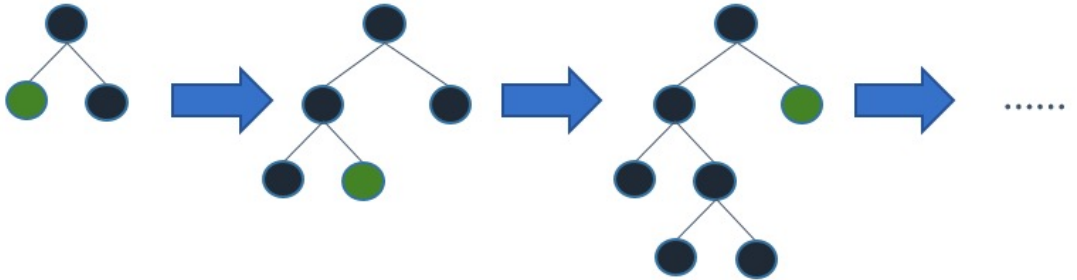
Level-wise growth splits nodes broadly at one depth. Leaf-wise growth chooses the current leaf with the largest available gain, so the tree can become asymmetric.

Work it out

Current candidate leaf	Best gain	Next action
Left shallow leaf	0.4	Wait
Right deeper leaf	1.8	Split this leaf
Another leaf	0.2	Wait

What this tells us. Leaf-wise growth can reduce training loss efficiently, but repeatedly splitting a small region may overfit. Limit the leaf count, depth, minimum leaf support, and regularization. LightGBM combines efficient histogram-based training with such growth controls; its name does not remove the need for validation.

Leaf-wise growth spends capacity where gain is largest



Leaf-wise tree growth

Walk through a leaf-wise tree with a leaf budget

Understand

Start from a root and permit at most four leaves. After the first split, the left leaf offers gain 0.4 and the right leaf offers 1.8.

Work through the example

Step	Largest available gain	Leaf count after split
Split root	Given useful root split	2
Split right leaf	1.8 instead of left 0.4	3
Split a right descendant	1.2 instead of left 0.4	4

Interpretation. The budget is now exhausted, even if other leaves still offer positive gains. Capacity has concentrated on the right branch, so depth can be uneven. Leaf count, depth, and minimum support impose different constraints; leaf-wise greediness alone is not an overfitting safeguard.

Categorical target statistics can leak the answer

Idea

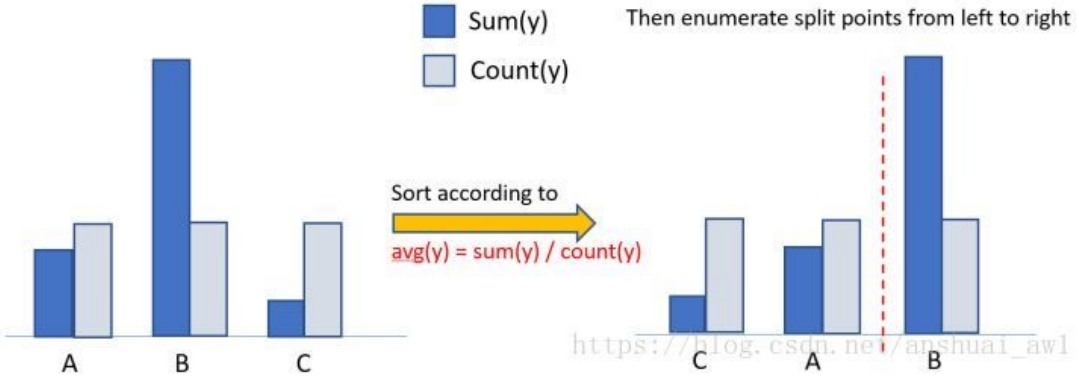
A category mean can be a useful numerical feature, but a row's own label must not determine its training encoding. For a rare category occurring once, an unprotected category mean equals its label.

Work it out

$$\text{Smoothed mean}(c) = \frac{\sum_{i \in c} y_i + a\pi}{n_c + a}.$$

What this tells us. CatBoost uses ordered strategies to reduce such target leakage and prediction shift; symmetric trees support efficient evaluation. A simplified teaching encoding uses only earlier rows in an order, plus a prior. Fit all encodings within training folds; smoothing alone does not erase self-label leakage.

Category statistics combine target sums and observation counts



https://blog.csdn.net/anshuai_awl

Calculate an ordered category statistic without using the current label

Understand

Three blue-category rows appear in order with labels 1, 0, 1. Use prior $\pi = 0.2$ and prior strength $a = 2$. Encode each row using earlier blue rows only.

Work through the example

Row	Previous positive sum / count	Encoding
1	0 / 0	$0.4/2 = 0.20$
2	1 / 1	$1.4/3 = 0.4667$
3	1 / 2	$1.4/4 = 0.35$

Interpretation. The current label is excluded from its own encoding. A full-category mean $2/3$ would use all three labels, including each row's own answer. This simplified ordered-statistic calculation illustrates a leakage boundary, not every detail of CatBoost training.

Compare the mechanism before comparing library names

Method	What changes between learners	Combination
Random forest	Bootstrap rows and candidate features	Average probabilities or values
AdaBoost	Observation attention	Weighted signed score
Gradient boosting	Loss-based correction targets	Add corrections
XGBoost / LightGBM	Efficient regularized tree fitting	Boosted score
Stacking	Different models or settings	Train a model on held-out predictions

Choose by the problem. Ask whether the limitation is unstable peers, missing residual structure, computational scale, or complementary model behavior. Validate the proposed improvement using a common split and metric.

A weak extra model can preserve accuracy but damage probabilities

Understand

Three illustrative models output probabilities for four cases with labels $(1, 0, 1, 0)$. Model C assigns low probability to every true label.

Work through the example

Case / label	Model A	Model B	Model C	Equal average
1 / 1	0.80	0.78	0.40	0.660
2 / 0	0.30	0.32	0.70	0.440
3 / 1	0.70	0.69	0.30	0.563
4 / 0	0.20	0.22	0.60	0.340

Interpretation. A and the equal average both classify all four cases correctly at 0.5, yet adding C reduces true-class confidence. On case 1, loss rises from $-\log 0.8 = 0.223$ to $-\log 0.66 = 0.416$. Complementarity and member quality should be assessed with the output metric you need.

Weighted voting is a fixed combination rule

Idea

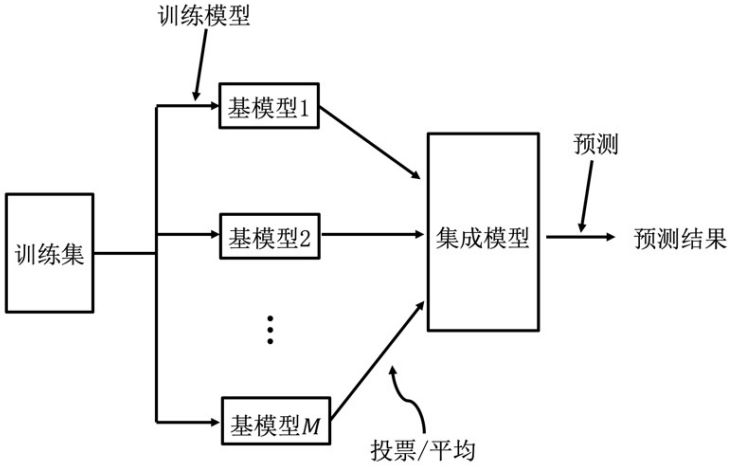
Two models predict failure probabilities 0.2 and 0.8. A fixed equal average is 0.5. A 3:1 weight favoring the first gives a different result.

Work it out

$$p_{\text{blend}} = \frac{3(0.2) + 1(0.8)}{3 + 1} = 0.35.$$

What this tells us. Weights may be selected on validation data, but they should reflect evidence rather than arbitrary preference. A stacking model learns a combination rule from base predictions and labels. Its training predictions must be produced without using each row in the corresponding base-model fit; otherwise the second layer learns from unrealistically easy examples.

Weighted voting is a fixed combination rule



Select a blend weight using a declared validation loss

Understand

Two validation cases have labels 1 and 0. Model A predicts $(0.9, 0.4)$ and model B $(0.6, 0.1)$. Blend with weight w on A and $1 - w$ on B.

Work through the example

Weight w	Blended probabilities	Mean log loss
0	0.60, 0.10	0.3081
0.25	0.675, 0.175	0.2927
0.50	0.75, 0.25	0.2877
0.75	0.825, 0.325	0.2927
1	0.90, 0.40	0.3081

Interpretation. Equal weighting is best on this small grid because the models'strengths complement each other. This is weight selection on validation data, so its resulting loss is selection evidence rather than an untouched final performance estimate.

Stacking trains on out-of-fold base predictions

Make realistic training inputs

Use three folds. A base model predicts each row only while that row is held out from its training data. Repeat this process for every base model.

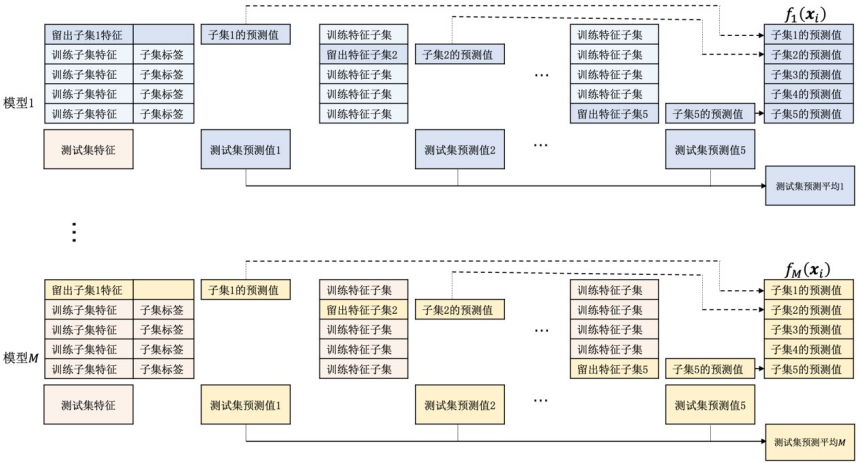
Fit the base model on	Predict held-out rows	Place predictions in
C, D, E, F	A, B	OOF rows A, B
A, B, E, F	C, D	OOF rows C, D
A, B, C, D	E, F	OOF rows E, F

Then fit the combination

Stitch each model's held-out predictions into one OOF column. Train the meta-model on these columns and the training labels.

Keep the boundary. Test labels never train either layer. Grouped or temporal tasks require matching fold boundaries.

Stacking trains on out-of-fold base predictions



Fit preprocessing inside each OOF training fold

Understand

OOF fold 1 holds out A and B. A has a missing sensor value; B has value 100. Training rows C, D, E, F have values 1, 2, 3, 4.

Work through the example

$$\text{Training-fold median} = 2.5, \qquad \text{All-row observed median} = 3.$$

Interpretation. For fold-1 predictions, fit the imputer on C–F and fill A with 2.5. Using all rows lets held-out B affect preprocessing. The same principle applies to target encodings and supervised feature selection. OOF exclusion concerns the entire base-learning pipeline, not only the final tree fit.

Six-row OOF example: each column is a new feature

Idea

Use three folds: A,B; C,D; E,F. For each fold, train each base model on the other four rows and predict the two held-out rows. The values below are illustrative OOF probabilities.

Work it out

Row	Held-out fold	Tree probability	Logistic probability	y
A	1	0.70	0.60	1
B	1	0.30	0.40	0
C	2	0.80	0.65	1
D	2	0.45	0.20	0
E	3	0.55	0.75	1
F	3	0.25	0.35	0

What this tells us. Fit a simple meta-model on the two probability columns and y. Each row is eligible because its base fits excluded that row. No test labels enter this table.

Regression stacking can combine complementary OOF residuals

Understand

Four training targets are $(10, 20, 30, 40)$. Two base models provide genuinely OOF predictions $(11, 19, 29, 41)$ and $(9, 21, 31, 39)$.

Work through the example

$$\hat{y}_{0.5} = \tfrac{1}{2}(\hat{y}_A + \hat{y}_B) = (10, 20, 30, 40).$$

Interpretation. Each base model has MSE 1 on these OOF rows; the equal combination has meta-training MSE 0. Complementary errors make a simple rule sufficient here. The second layer has still used these labels to choose a combination, so zero on this table is not proof of zero error for an untouched test population.

In-sample stacking teaches the meta-model a false lesson

Idea

A deep tree can nearly memorize the training labels. Compare its apparent training probabilities with genuinely held-out probabilities.

Work it out

Evaluation source	Tree probability on a positive case	What meta-model observes
Same rows used to fit tree	0.999	Apparently almost perfect
Rows excluded from base fit	0.60	Realistic uncertainty

What this tells us. Training the meta-model on in-sample probabilities rewards memorization and can overwhelm genuinely useful complementary models. OOF construction fixes the base-prediction boundary, but evaluating the complete stack still needs held-out data. Repeatedly searching stacks on the final test set also overfits evaluation.

Compare the illusion of in-sample loss with OOF loss

Understand

On one positive and one negative case, a memorizing tree returns training probabilities $(0.999, 0.001)$ but OOF probabilities $(0.6, 0.4)$. A simpler model gives OOF probabilities $(0.7, 0.3)$.

Work through the example

$$L_{\text{tree,in}} = -\log 0.999 \approx 0.001, \quad L_{\text{tree,OOF}} = -\log 0.6 = 0.511.$$
$$L_{\text{simple,OOF}} = -\log 0.7 = 0.357.$$

Interpretation. The apparently perfect tree is actually worse on the appropriate OOF comparison here. A meta-model trained on in-sample confidence would receive misleading evidence. Use comparable unseen predictions before judging which base learner deserves influence.

Prediction must recreate the meta-model inputs

Idea

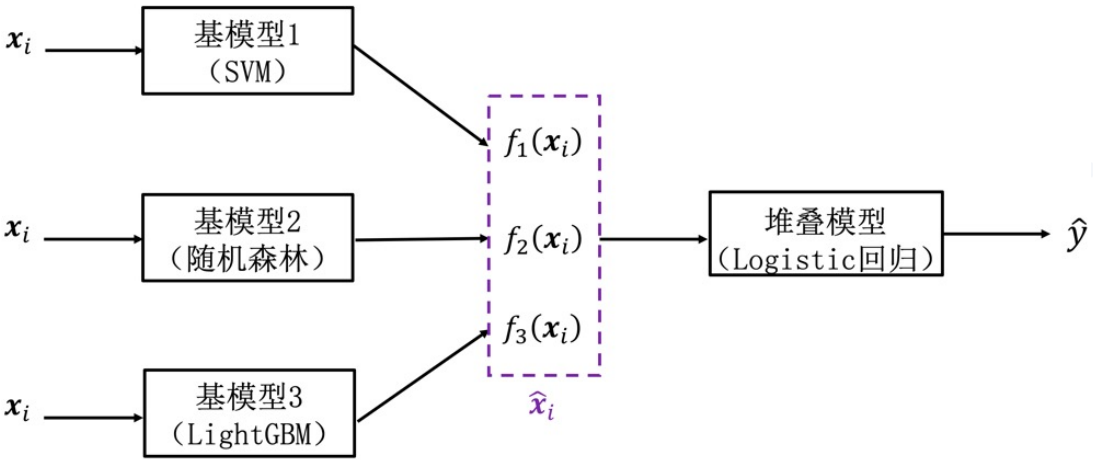
After fitting the meta-learner, produce new-case base predictions using a defined strategy: either average the fold models or refit each base learner on all training data. Follow the chosen implementation consistently.

Work it out

$$z_{\text{new}} = (p_{\text{tree}}, p_{\text{logistic}}), \qquad p_{\text{final}} = \sigma(\beta_0 + \beta_1 p_{\text{tree}} + \beta_2 p_{\text{logistic}}).$$

What this tells us. Example: $z = (0.70, 0.60)$ and coefficients $(-2, 2, 1)$ give score $-2 + 1.4 + 0.6 = 0$, hence probability 0.5. OOF columns define the meta-training format; preserve feature order, class meaning, and the same preprocessing at inference.

Prediction must recreate the meta-model inputs



Recreate meta-features from fold models at inference

Understand

For a new device, the three fold-trained tree models predict $(0.6, 0.7, 0.8)$ and the three logistic models $(0.5, 0.6, 0.7)$. Average within each base family.

Work through the example

$$z = (0.7, 0.6), \qquad F_{meta} = -2 + 2(0.7) + 1(0.6) = 0, \quad p_{meta} = 0.5.$$

Interpretation. If full-data refits instead output $(0.8, 0.65)$, the same meta-model gives score 0.25 and probability 0.562. Averaging fold models and refitting are different inference strategies. Follow the selected implementation and validation procedure consistently, while preserving class meaning and column order.

Reconstruct the correction, then defend the combination

Question	What a complete answer includes
AdaBoost: one wrong weight 0.2	Error 0.2, $\alpha = 0.6931$, normalize to wrong mass 0.5
Regression: $y = 10$, current $F = 6$	Squared-loss residual $+4$; add a learned correction
Binary classification: $y = 1$, $p = 0.2$	Negative gradient $+0.8$; update score, then sigmoid
Stacking: deep-tree training predictions	Replace them with OOF predictions before meta-training

Bridge to neural networks. The loss continues to define what should improve. Boosting adds a new function each round; neural-network training adjusts the parameters of an existing differentiable function.

Final exercise: distinguish four correction signals

Understand

Give a complete numerical answer for four cases: AdaBoost error 0.2; squared-loss target 10 with prediction 6; binary target 1 with probability 0.2; and a stacking row seen by its base learner.

Work through the example

$$\alpha = \frac{1}{2}\log 4 = 0.6931, \qquad r_{square} = 4, \qquad r_{logit} = 0.8.$$

Interpretation. AdaBoost updates observation attention; regression boosting learns a cost correction; logit boosting updates a score before converting to probability; stacking requires an OOF replacement for the in-sample base prediction. Explain each target and combination rule, then connect loss-directed improvement to the parameter updates of neural networks in the next lecture.